

IFT436 - Algorithmes et structures de données

Devoir #2

Question 1 Une question d'échauffement

(a) On démontre les deux implications.

$\Rightarrow$ Supposons que G est fortement connexe. Pour tout sommet $u \in V$ et tout sommet $v \in V$, il existe un chemin orienté de u à v dans G . Ainsi,

$$r(G, u) = V.$$

Il existe également un chemin orienté de v à u dans G . En renversant chacune de ses arêtes, on obtient un chemin orienté de u à v dans G^T . Par conséquent,

$$r(G^T, u) = V.$$

Les deux égalités sont donc vérifiées pour tout $u \in V$.

$\Leftarrow$ Supposons maintenant que, pour tout $u \in V$,

$$r(G, u) = V \quad \text{et} \quad r(G^T, u) = V.$$

Soient $u, v \in V$ deux sommets quelconques. Puisque $v \in r(G, u)$, il existe un chemin orienté de u à v dans G . De plus, puisque $v \in r(G^T, u)$, il existe un chemin orienté de u à v dans G^T . En renversant ses arêtes, on obtient un chemin orienté de v à u dans G .

Il existe donc un chemin orienté dans les deux sens entre toute paire de sommets. Ainsi, G est fortement connexe, ce qui complète la démonstration.

(b) Soit $u \in V$ un sommet tel que

$$r(G, u) = r(G^T, u) = V.$$

Montrons d'abord que G est fortement connexe. Soient $v, w \in V$ deux sommets quelconques.

Puisque $v \in r(G^T, u)$, il existe un chemin orienté de u à v dans G^T , donc un chemin orienté de v à u dans G . De plus, puisque $w \in r(G, u)$, il existe un chemin orienté de u à w dans G .

En passant par u , on peut donc aller de v à w . Comme v et w sont quelconques, G est fortement connexe. D'après le résultat de (a), on obtient alors

$$\boxed{r(G, v) = r(G^T, v) = V \quad \text{pour tout } v \in V.}$$

(c) On suppose que le graphe est représenté par des listes d'adjacence. L'algorithme est le suivant :

- i Choisir un sommet arbitraire $u \in V$.
- ii Effectuer un parcours en largeur (BFS) dans G à partir de u . Si tous les sommets ne sont pas visités, retourner **faux**.
- iii Construire le graphe transposé G^T en renversant chaque arête de G .
- iv Effectuer un BFS dans G^T à partir du même sommet u . Si tous les sommets sont visités, retourner **vrai** ; sinon, retourner **faux**.

Les deux parcours permettent de déterminer respectivement $r(G, u)$ et $r(G^T, u)$. D'après (b), si ces deux ensembles sont égaux à V , alors G est fortement connexe. Réciproquement, d'après (a), si G est fortement connexe, les deux parcours visitent tous les sommets. L'algorithme retourne donc **vrai** si et seulement si G est fortement connexe.

Complexité. Comme vue en classe, Chaque BFS visite chaque sommet au plus une fois et examine chaque arête au plus une fois. Il coûte donc $O(|V| + |E|)$. La construction de G^T consiste à créer une liste d'adjacence pour chaque sommet et à parcourir chaque arête (v, w) pour ajouter l'arête (w, v) . Elle coûte également $O(|V| + |E|)$.

La complexité totale est donc

$$O(|V| + |E|).$$

Je pense que cette complexité est optimale dans le pire cas pour une représentation par listes d'adjacence car il peut être nécessaire d'examiner tout les sommets et d'arêtes pour décider de la forte connexité.

- (d) L'idée est de trouver la composante fortement connexe contenant un sommet u en effectuant deux parcours en largeur (BFS).

Un BFS dans G à partir de u trouve les sommets accessibles depuis u , soit $r(G, u)$. Un BFS dans G^T à partir du même sommet trouve les sommets qui peuvent atteindre u dans G , soit $r(G^T, u)$. Leur intersection

$$X = r(G, u) \cap r(G^T, u)$$

contient donc exactement les sommets qui sont mutuellement accessibles avec u .

On enregistre cette composante, puis on recommence avec un sommet qui n'appartient à aucune composante déjà trouvée. Le graphe transposé est construit une seule fois au début.

Pseudo-code.

On suppose que $\text{BFS}(G, u)$ retourne l'ensemble des sommets visités.

COMPOSANTESFORTEMENTCONNEXES(G)

```

    Construire  $G^T$ 
    composantes  $\leftarrow$  liste vide
    non_classes  $\leftarrow V(G)$ 
    Tant que non_classes  $\neq \emptyset$  :
        Choisir  $u \in$  non_classes
         $X \leftarrow \text{BFS}(G, u) \cap \text{BFS}(G^T, u)$ 
        composantes.append( $X$ )
        non_classes  $\leftarrow$  non_classes  $\setminus X$ 
    Retourner composantes

```

Pour deux sommets $v, w \in X$, il existe un chemin de v à u et un chemin de u à w dans G . Tous les sommets de ces chemins sont eux aussi mutuellement accessibles avec u , donc appartiennent à X . On peut ainsi aller de v à w entièrement dans $G[X]$. Par conséquent, $G[X]$ est fortement connexe.

De plus, si un ensemble non vide $Z \subseteq V \setminus X$ pouvait être ajouté à X en conservant la forte connexité, chaque sommet de Z serait accessible depuis u et pourrait atteindre u . Ces sommets appartiendraient donc déjà à X , ce qui est une contradiction. Ainsi, X est maximal.

Chaque itération trouve donc une composante fortement connexe complète. Comme on choisit ensuite un sommet non classé, aucune composante n'est retournée deux fois. À la fin, tous les sommets sont classés, donc toutes les composantes ont été trouvées.

Complexité. Posons $n = |V|$ et $m = |E|$. Comme discuter en (c), avec des listes d'adjacence, la construction de G^T coûte $O(n + m)$. Chaque itération effectue un BFS dans G et un BFS dans G^T , chacun de coût $O(n + m)$. L'intersection des deux ensembles de X et X^t des sommets visités coûte $O(n)$: on parcourt les n sommets et on conserve ceux qui sont marqués comme visités dans les deux tableaux.

Puisque $u \in X$, chaque itération classe au moins un nouveau sommet. Il y a donc au plus n itérations, pour une complexité totale

$$O(n(n + m)).$$

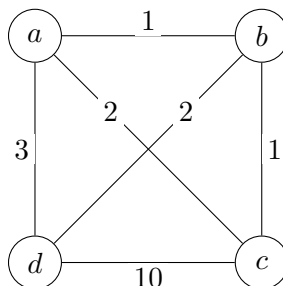
L'algorithme est donc polynomial !

Question 2 Le problème de la sous-somme

(a)

Question 3 Puissances en temps logarithmique

- (a) On donne un contre-exemple sur un graphe complet à quatre sommets. Les coûts des arêtes sont représentés ci-dessous.

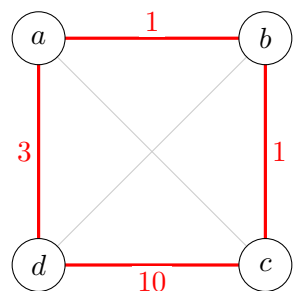


Supposons que l'algorithme commence au sommet a . Il choisit d'abord b , car $f(ab) = 1$ est le coût minimal parmi les arêtes reliant a aux autres sommets. Depuis b , il choisit ensuite c , puisque $f(bc) = 1 < f(bd) = 2$. Il ne reste alors que d , puis l'algorithme revient à a . Le cycle retourné est donc

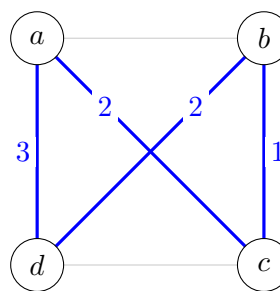
$$C_{\text{glouton}} = (a, b, c, d, a), \quad f(C_{\text{glouton}}) = 1 + 1 + 10 + 3 = 15.$$

Pourtant, le cycle $C' = (a, c, b, d, a)$ a un coût inférieur :

$$f(C') = 2 + 1 + 2 + 3 = 8 < 15.$$



Cycle glouton : coût 15



Autre cycle : coût 8

L'algorithme peut donc retourner un cycle qui n'est pas de coût minimum. Il ne résout pas correctement le problème du commis voyageur : ses choix localement les moins coûteux ne garantissent pas un coût total minimal.